

Wyatt Corcoran
Date: 9/17/2026\

Prompt:

Objective

The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap Modules.

Make the program as compact and simplified as possible, write in C or CPP

Test File format:

Command Description

EMAIL <sender category>,
<subject line>,
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- Sample Test File:

EMAIL Peer, Can you help me on this?, 12-01-2024

EMAIL OtherPerson, Try our product, 12-19-2024

EMAIL Boss, Important, 12-20-2024

EMAIL Subordinate, How do I handle this?, 12-25-2024

EMAIL ImportantPerson, Health Insurance Enrollment, 12-31-2024

EMAIL Boss, Never Mind, 01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

AI code analysis:

- a. Claude by Anthropic and Gemini by Google
- b. Both accessed via google
- c. Prompt above
- d. Both outputs are in .txt files in the GitHub repo
- e. Both codes work; however Claude takes many extra steps. The space/time complexity is comparable; however, the code by Gemini uses far fewer calls and has fewer libraries called. Maintainability-wise, Gemini wins for being more readable; however, Claude has more comments. The comments are only slightly helpful from either.
- f. I selected Gemini since it uses fewer libraries and has far more concise functions. It was far easier to read and make adjustments to for example fixing up the edge cases and commenting was far easier since it used simpler methods.
- g. My changes:
Fixed a logical error on line 25 that made the code accidentally prioritize the oldest email for tiebreakers. Added 2 instances of feedback if the edge case is hit, printing inbox empty if the user tries to read or pop from an empty inbox. Added an if statement around the email input requirements as well as &&'s to make sure all requirements of the email structure are met. These changes fix correctness by properly covering edge cases with the user. I also fixed logical bugs left in the original code as well as preventing bad input.